STUDENT PERFORMANCE & ATTENDANCE ANALYSIS SYSTEM

A PROJECT REPORT

Submitted by

Roll Number	Registration Number	Student Code	Student Name
26102410007	26102000127	BWU/MCA/26/009	Debajyoti Bayen

in partial fulfillment for the award of the degree Of

MASTER OF COMPUTER APPLICATIONS

IN

DEPARTMENT OF COMPUTATIONAL SCIENCES



BRAINWARE UNIVERSITY

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata - 700 125



BRAINWARE UNIVERSITY

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata - 700 125

[COMPUTATIONAL SCIENCES]

BONAFIDE CERTIFICATE

Certified that this project report "Student Performance & Attendance Analysis System" is the bonafide work **of "Debajyoti Bayen"** who carried out the project work under my supervision.

_ _ _ _ _ _ _ _ _ _ _ _	_ _ _ _ _ _ _ _ _ _ _ _
SIGNATURE	**SIGNATURE**
DR. Jayanta Aich	**MR. Rahul Kumar Ghosh**
HEAD OF THE DEPARTMENT	***SUPERVISOR***
Computational Sciences	**Assistant Professor**
Brainware University	**Computational Sciences**

ABSTRACT

In contemporary academic environments, managing student academic progression alongside cumulative attendance compliance forms an essential administrative pillar. Traditional manual logging methods—reliant on physical paper registers or unlinked spreadsheets—frequently introduce calculation discrepancies, lack instant validation safeguards, and struggle to deliver actionable visual insights.

To resolve these operational limitations, this project presents the *Student Performance & Attendance Analysis System*, a lightweight, responsive web-based platform developed using Python and the Flask micro-framework. The system incorporates an automated computational engine that calculates total marks (out of 400), subject averages, overall percentages, and standardized letter grades (ranging from A+ to F) across four technical modules: Python, C Programming, Database Management Systems (DBMS), and Web Development. To support timely institutional intervention, an automated filtering pipeline monitors student attendance against the mandatory 75% threshold, immediately flagging defaulters on an active alert roster.

The application leverages a dual-layer, zero-configuration persistence model combining atomic Comma Separated Values (students.csv) storage with a mirrored plain text audit trail (students.txt), eliminating the need for external database servers while ensuring platform portability and data integrity. Furthermore, built-in algorithmic routines provide descending bubble-sort ranking by total marks and substring searching by student roll or name. Data exploration is reinforced through server-side statistical aggregation with Pandas and graphical rendering via Matplotlib, dynamically generating comparative Bar Charts, Percentage Trajectory Line Graphs, and Performance Distribution Histograms. Tested across an active cohort, the system demonstrates high computational reliability, robust validation enforcement, and intuitive analytical clarity for faculty evaluation workflows.

Keywords: Educational Data Analytics, Flask Web Framework, Academic Performance Evaluation, Attendance Defaulter Monitoring, Pandas Data Aggregation, Matplotlib Visualization, CSV Persistence.

TABLE OF CONTENTS

Section	Topic / Chapter Description	Page Ref.
1	Introduction	5
2	Objective	6
3	Planning	7
4	Requirement Analysis	9
5	System Flow	11
	5.1 Context Level Diagram	11
	5.2 Data Flow Diagram Level 0	12
	5.3 Data Flow Diagram Level 1	13
	5.4 Use Case Diagram	14
	5.5 Activity Diagram	15
	5.6 Sequence Diagram	16
	5.7 Flow Chart	17
6	Proposed Design	18
7	Experimental Result	19
	7.1 Data Loading Stage	19
	7.2 Performance Analysis	20
	7.3 Attendance Analysis	21
	7.4 Bar Chart Generation	22
	7.5 Line Graph Generation	23
	7.6 Histogram Generation	24
8	Future Scope	25
9	Conclusion	26

1. INTRODUCTION

In modern educational institutions, managing and evaluating student academic progress alongside daily attendance records is one of the most critical responsibilities of faculty members and academic administrators. Traditionally, this process has been handled using manual paper registers, static desktop spreadsheets, or fragmented office files. However, manual record-keeping suffers from several persistent vulnerabilities, including computational errors during percentage and grade determination, inadvertent duplicate roll number entries, lack of automated defaulter detection for low attendance, and an absence of interactive visual analysis.

To overcome these challenges, I have developed the Student Performance & Attendance Analysis System. This project is a self-contained, responsive, web-based analytical tool engineered using Python, the lightweight Flask WSGI micro-framework, Pandas for statistical aggregation, NumPy for numerical operations, and Matplotlib for server-side chart rendering. The front end is developed using responsive HTML5, modern CSS styling, and asynchronous JavaScript (AJAX/Fetch API) to enable real-time modal updates and chart rendering without tedious page reloads.

Unlike overly complex enterprise platforms that demand heavy relational database server installations (such as MySQL or PostgreSQL) and complex configuration overhead, this application uses a structured Comma Separated Values (CSV) persistence architecture (`data/students.csv`) paired with an automatically synchronized plain text audit log (`data/students.txt`). This lightweight file-based data layer ensures zero-configuration deployment, complete portability across computing platforms, and instantaneous data transparency.

The system provides complete CRUD (Create, Read, Update, Delete) capability for student academic records covering four major technical subjects—Python, C Programming, Database Management Systems (DBMS), and Web Development —alongside each student's overall attendance percentage. Beyond simple record keeping, the system automatically computes aggregate totals (out of 400 marks), subject averages, overall percentages, letter grades (A+ to F), highlights students failing to meet mandatory attendance thresholds (< 75%), performs bubble-sort ranking by total marks, supports real-time keyword substring filtering by name or roll number, and renders interactive analytical visualizations (Bar Charts, Line Graphs, and Histograms).

2. OBJECTIVE

The primary objective of this project is to provide a clean, reliable, and user-friendly web-based platform that automates the end-to-end management of student marks and attendance data while providing immediate academic insights through descriptive statistics and visualization.

The specific operational and technical goals achieved in this project include:

- **Automated Academic Computations:** Eliminating manual arithmetic errors by automatically calculating Total Marks (out of 400), Subject Averages, Percentage scores, and accurate Letter Grades (A+, A, B, C, D, F) based on standardized institutional evaluation rules.
- **Proactive Attendance Defaulter Monitoring:** Providing real-time visual alerts and filtered categorization for students possessing an attendance rate below the mandatory 75% threshold, enabling immediate faculty intervention before university examinations.
- **Strict Data Validation & Integrity Enforcement:** Implementing robust server-side sanitization routines that prevent duplicate roll numbers, restrict names to alphabetical characters and spaces, enforce mark boundaries strictly between 0 and 100, and ensure attendance values remain valid percentages.
- **Efficient Querying, Sorting & Search Routines:** Providing fast search facilities that allow instructors to locate students by full or partial roll number and name strings, alongside a descending Bubble Sort mechanism that immediately ranks the entire class from highest to lowest total score.
- **Integrated Statistical Aggregation via Pandas:** Computing high-level academic metrics—including class highest score, lowest score, mean total marks, class-wide average percentage, and overall average attendance—with a single click via asynchronous API endpoints.
- **Server-Side Graphical Visualization:** Generating publication-grade visual charts (Bar Charts of total marks, Line Graphs of individual percentages, and Frequency Histograms of performance brackets) using Matplotlib and streaming them directly to the client browser.

3. PLANNING

Effective software engineering demands a structured planning phase to ensure that functional goals align with technical feasibility, system constraints, and development timelines. During the planning stage of this project, I conducted a comprehensive feasibility analysis and adopted an iterative development methodology.

3.1 Feasibility Study

- **Technical Feasibility:** The project leverages Python 3, a highly stable and versatile programming language. Flask was chosen because it is lightweight, requires minimal boiler-plate setup, and includes native Jinja2 template rendering. Supporting libraries including Pandas, NumPy, and Matplotlib are industry standards for data science. Because the application runs in any standard modern web browser and requires no specialized graphics acceleration or external database servers, technical feasibility was established at 100%.
- **Operational Feasibility:** Faculty members and teachers often experience fatigue when navigating cluttered enterprise portals. This system was designed with an intuitive, clean, single-dashboard interface where data entry, records tables, search bars, defaulter lists, and chart triggers reside in a single cohesive view. No specialized technical training is required to operate the software.
- **Economic Feasibility:** The entire software stack is built upon free, open-source technologies (Python, Flask, Pandas, Matplotlib, HTML5/CSS3). There are no licensing fees, proprietary database runtime costs, or third-party API subscription expenses, making the project exceptionally cost-effective.

3.2 Software Development Life Cycle (SDLC) Methodology

The project followed the classical Waterfall Model with iterative validation loops across distinct milestones:

- **Phase 1 - Problem Definition & Scope:** Gathering functional needs from academic record management workflows and defining schema attributes (Roll, Name, 4 Subject Marks, Attendance).
- **Phase 2 - Architectural & Algorithm Design:** Drafting system flows, data flow diagrams, UML models, validation logic, and sorting algorithms.
- **Phase 3 - Implementation & Module Coding:** Developing the Flask backend (`app.py`), establishing CSV and TXT synchronization routines, designing the Jinja2 front-end (`templates/index.html`), and configuring Matplotlib headless chart rendering (`matplotlib.use('Agg')`).
- **Phase 4 - Testing & Defect Rectification:** Executing boundary tests (negative marks, non-alphabetic names, duplicate roll numbers), checking attendance filtering logic, and validating chart generation pipelines.
- **Phase 5 - Deployment & Documentation:** Verifying file system portability, bundling source archives, and compiling this comprehensive technical report.

Project Milestone	Key Tasks & Deliverables	Timeline	Status
Phase 1: Inception & Analysis	Requirement gathering, schema definition, setup	Week 1 - 2	Completed
Phase 2: Architectural Design	DFD, UML modeling, system logic flowchart	Week 3	Completed
Phase 3: Core Implementation	Flask routes, CRUD operations, CSV/TXT persistence	Week 4 - 5	Completed
Phase 4: Analytics & Charts	Pandas aggregation, Matplotlib bar/line/histogram	Week 6	Completed
Phase 5: UI/UX & Testing	AJAX chart modals, sorting/search, validation tests	Week 7	Completed
Phase 6: Final Documentation	Technical write-up, architectural diagrams, report	Week 8	Completed

4. REQUIREMENT ANALYSIS

A rigorous requirement analysis was conducted to establish the operational boundaries and technical specifications necessary for seamless deployment and execution of the application.

4.1 Hardware Requirements

- **Processor:** Intel Core i3 / AMD Ryzen 3 or higher (minimum 1.8 GHz clock speed).
- **System RAM:** Minimum 2 GB RAM (4 GB recommended for smooth Pandas DataFrame operations and rapid Matplotlib image rendering).
- **Storage Space:** Minimum 150 MB free disk space for Python runtime environment, application dependencies, CSV datasets, and generated chart PNGs.
- **Display Resolution:** 1366 x 768 pixels or higher for optimal viewing of the web dashboard and graphical visualizations.

4.2 Software Requirements

- **Operating System:** Cross-platform compatibility across Windows 10/11, Ubuntu Linux (20.04/22.04 LTS), macOS, or any standard POSIX-compliant environment.
- **Programming Environment:** Python 3.8 to Python 3.12+ runtime.
- **Core Web Framework:** Flask (WSGI compliant web application framework).
- **Data Science & Visualization Libraries:** Pandas (version 1.3+), NumPy (version 1.20+), Matplotlib (version 3.4+ configured with the headless `Agg` backend).
- **Client Web Browser:** Google Chrome, Mozilla Firefox, Microsoft Edge, or Safari with JavaScript enabled.

4.3 Functional Requirements

- **Student Record Creation:** Accept roll number, full student name, marks across Python, C Programming, DBMS, Web Development, and attendance percentage.
- **Data Sanitization:** Reject submissions with duplicate roll numbers, non-alphabetic names, mark values outside 0-100, or attendance values outside 0-100.
- **Record Modification & Deletion:** Allow updating existing student details while preventing roll conflicts; permit immediate deletion with automatic cascade updating to both CSV and TXT files.
- **Academic Computation Engine:** Compute Total Marks ($Python + C + DBMS + Web$), Average Marks ($Total / 4$), Percentage ($(Total / 400) \times 100$), and assign Grades (A+, A, B, C, D, F).
- **Attendance Defaulter Flagging:** Identify all students possessing attendance strictly below 75.0% and present them in a dedicated warning list.
- **Sorting & Filtering:** Sort all students descending by Total marks using an in-memory bubble sort; filter students matching roll or name substring queries.
- **Statistical Analysis & Visualization:** Deliver class-level analytics via Pandas; generate Bar Charts, Line Graphs, and Histograms on demand.

4.4 Non-Functional Requirements

- **Performance & Responsiveness:** Calculations and file read/write operations execute in sub-millisecond timeframes for standard cohort sizes (up to several thousand records).
- **Reliability & Fault Tolerance:** Data is dual-persisted to both CSV and TXT; form inputs are validated before file mutation to prevent corrupted records.
- **Maintainability & Readability:** Clean, modular Python code structure with clear function names, explicit comments, and decoupled front-end Jinja templates.
- **Portability:** Self-contained file-based database means the entire application folder can be moved to any computer running Python without database reconfiguration.

5. SYSTEM FLOW

System flow diagrams provide an architectural blueprint illustrating how information enters the application, moves through various processing and computational stages, interacts with persistent file stores, and returns meaningful analytical responses to the end user. The following sections provide detailed architectural models and UML representations corresponding to the implementation.

5.1 Context Level Diagram (DFD Level 0 Context)

The Context Level Diagram represents the highest conceptual abstraction of the application. It models the entire system as a single process (Process 0.0) and highlights the data exchanges between the external entity (Faculty / Administrator), the core system, and the underlying storage repositories.

In this architecture, the faculty member provides raw input data consisting of student identity credentials (Roll Number, Full Name), numerical subject marks across four academic modules, attendance percentages, and operational queries (search keywords, sort requests, analysis triggers). The system evaluates these inputs, sanitizes them against business rules, and communicates with the storage layer (`students.csv` and `students.txt`). In return, the system generates evaluated grade reports, highlights attendance defaulters (< 75%), provides aggregated statistical metrics, and streams visual charts.

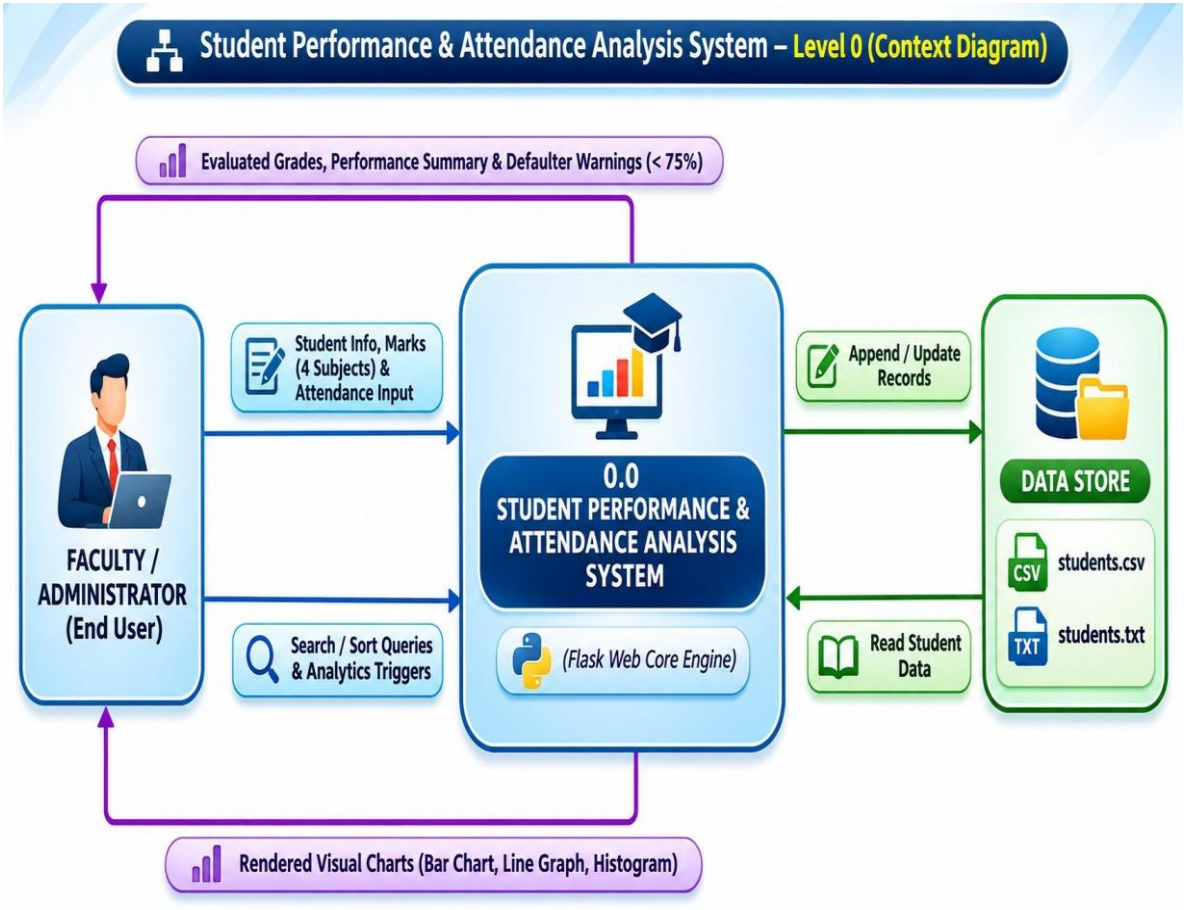


Figure 5.1: Context Level Diagram (DFD Level 0 Context)

5.2 Data Flow Diagram Level 0

The Data Flow Diagram Level 0 decomposes the overarching system into four major operational subsystems, illustrating internal data flow paths and storage interactions:

- **Process 1.0 (Student Entry & Validation Engine):** Receives raw form submissions from the user, validates field constraints, inspects existing records to ensure roll uniqueness, and updates the CSV and TXT files.
- **Process 2.0 (Calculation & Grading Engine):** Retrieves raw student records, calculates total marks, subject averages, percentages, assigns standardized letter grades, and returns formatted data tables.
- **Process 3.0 (Search, Sort & Attendance Filter):** Handles real-time search queries and bubble sort ranking routines, extracting records from `students.csv` and isolating defaulters.
- **Process 4.0 (Pandas Analytics & Matplotlib Charts):** Transforms student records into a Pandas DataFrame to compute high-level statistics and invokes Matplotlib to plot and save PNG charts.

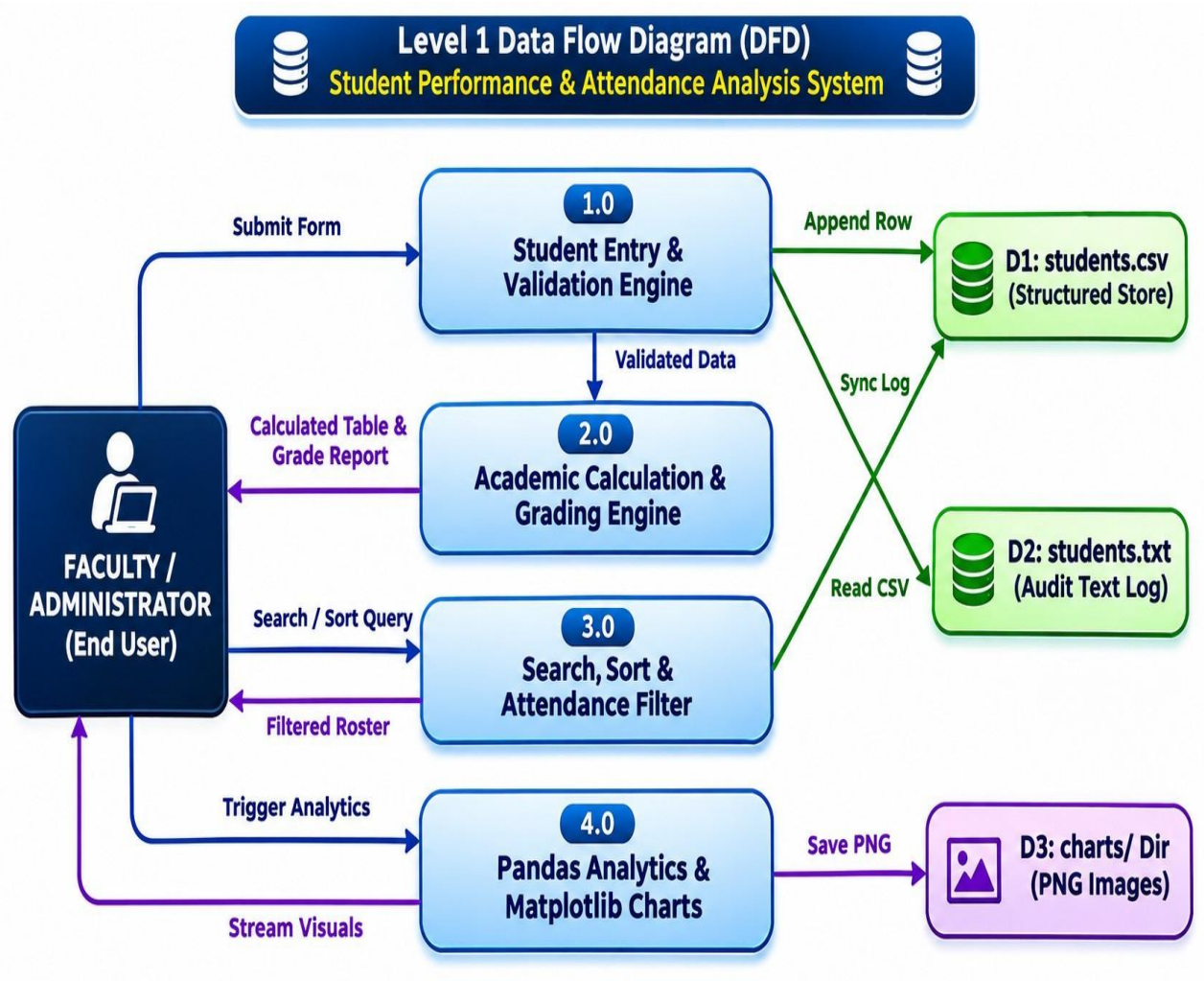


Figure 5.2: Data Flow Diagram (DFD Level 0 - System Subsystems)

5.3 Data Flow Diagram Level 1

The DFD Level 1 provides a granular decomposition of each subsystem into its underlying algorithmic components, tracing input flow directly to programmatic routines in `app.py`:

- **Sub-Process 1.1 & 1.2:** Input form validation checking alphabetic character constraints, integer range boundaries (0-100), float attendance validation, roll duplication checks, and atomic append operations to `students.csv`.
- **Sub-Process 2.1 & 2.2:** Execution of `calculate_total()`, `calculate_average()`, `calculate_percentage()`, and conditional grade evaluation (`calculate_grade()`).
- **Sub-Process 3.1 & 3.2:** Filtering loop for attendance strictly under 75%, substring matching for search, and nested iteration for descending bubble sort.
- **Sub-Process 4.1 & 4.2:** Pandas column aggregations (`max()`, `min()`, `mean()`), followed by Matplotlib plotting routines (`plt.bar()`, `plt.plot()`, `plt.hist()`) and file streaming.

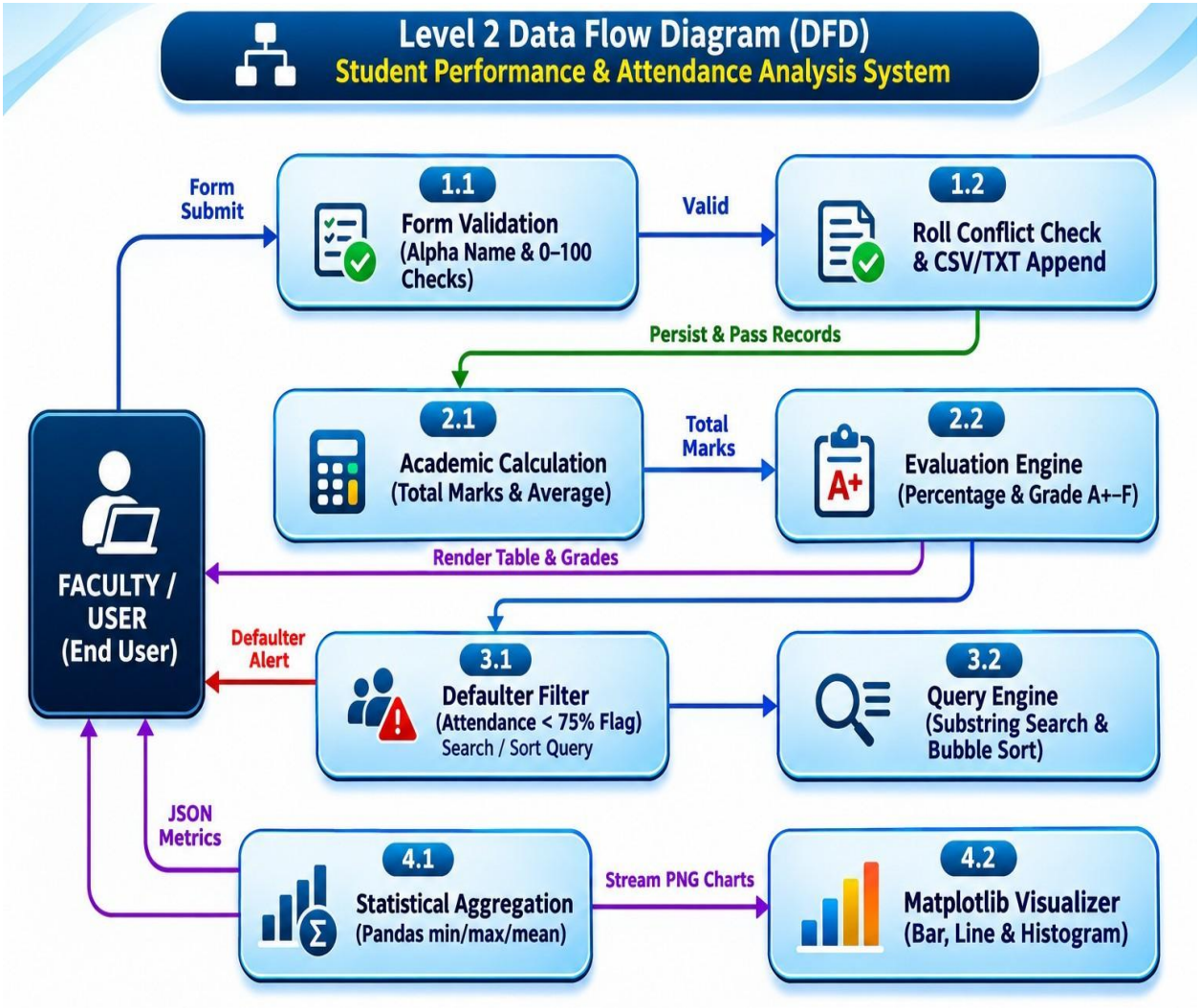


Figure 5.3: Detailed Data Flow Diagram (DFD Level 1 - Granular Routines)

5.4 Use Case Diagram

The Use Case Diagram defines the functional boundary of the system and models the interactions between the primary actor (Faculty / Administrator) and the various system capabilities.

Key use cases include Add Student Record, Edit Student Details, Delete Student, View Student Table, Filter Low Attendance, Search Records, Sort Students, Run Statistical Analysis, and Generate Charts. Critical operations like input validation and grade calculation are modeled with `<<include>>` relationships, ensuring that record insertion or table rendering cannot proceed without executing validation and calculation routines.

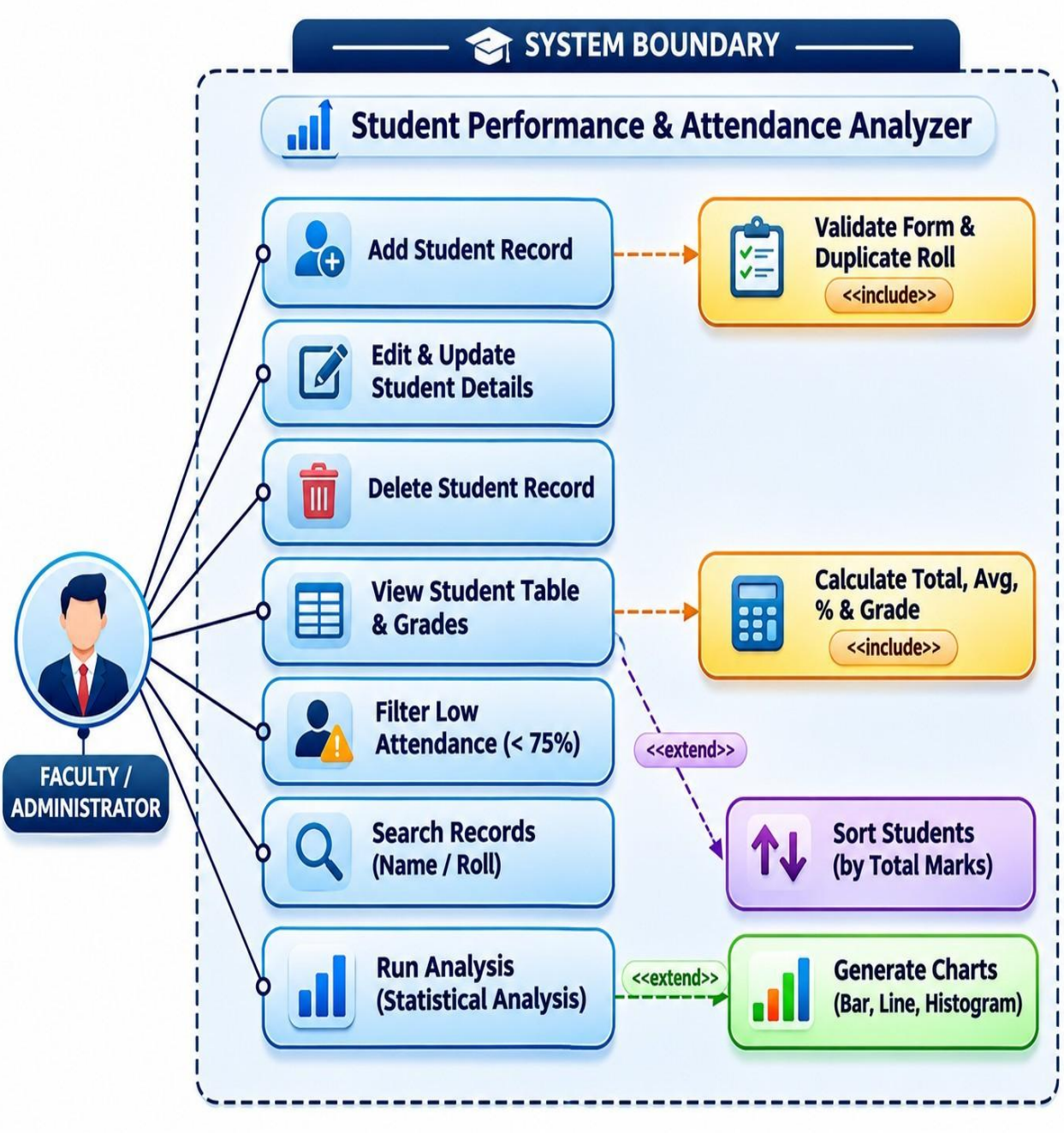


Figure 5.4: UML Use Case Diagram of the System

5.5 Activity Diagram

The UML Activity Diagram models the dynamic workflow and procedural control flow of the application from session initialization to termination.

Upon opening the dashboard (`GET /`), the application loads `students.csv`, computes academic metrics, checks attendance levels, and renders the primary dashboard. The user then reaches an operational decision diamond where they can choose to add/update records, delete a student, trigger a search or sort query, or request statistical analytics and graphical plots. All branches execute their respective backend routines, synchronize the storage files, merge at a synchronization bar, and refresh the browser interface.

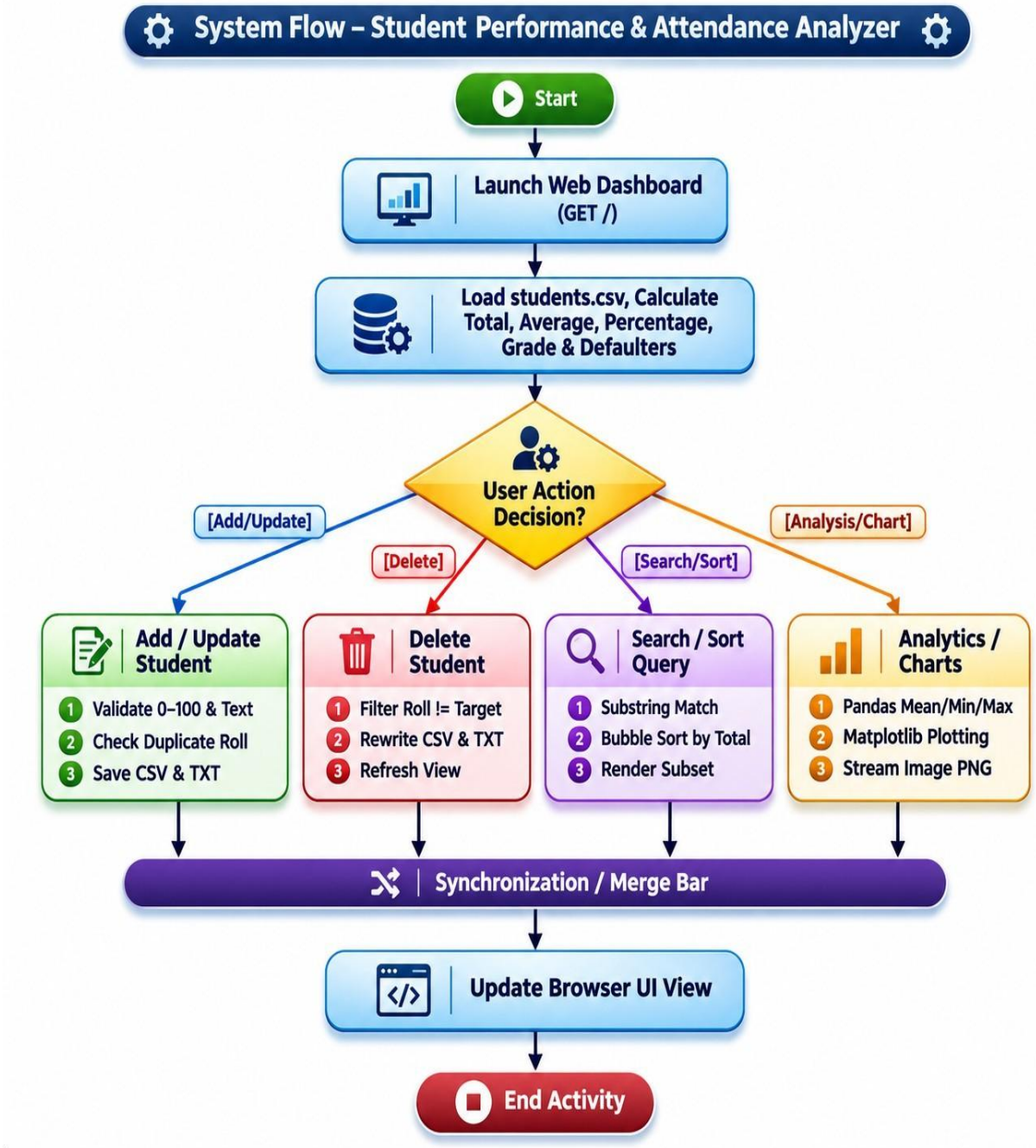


Figure 5.5: UML Activity Diagram Depicting Procedural Workflow

5.6 Sequence Diagram

The UML Sequence Diagram illustrates the chronological sequence of method invocations and HTTP request-response cycles occurring between the system components: User, Web Browser (`index.html`), Flask Controller (`app.py`), File Storage (`students.csv`/`students.txt`), and the Pandas/Matplotlib Analytics engine.

The diagram highlights synchronous HTTP POST submissions for record creation, internal execution of `validate_student_values()`, file writing, asynchronous AJAX queries for Pandas analysis, and binary file streaming of rendered Matplotlib PNG images via `send_file()`.

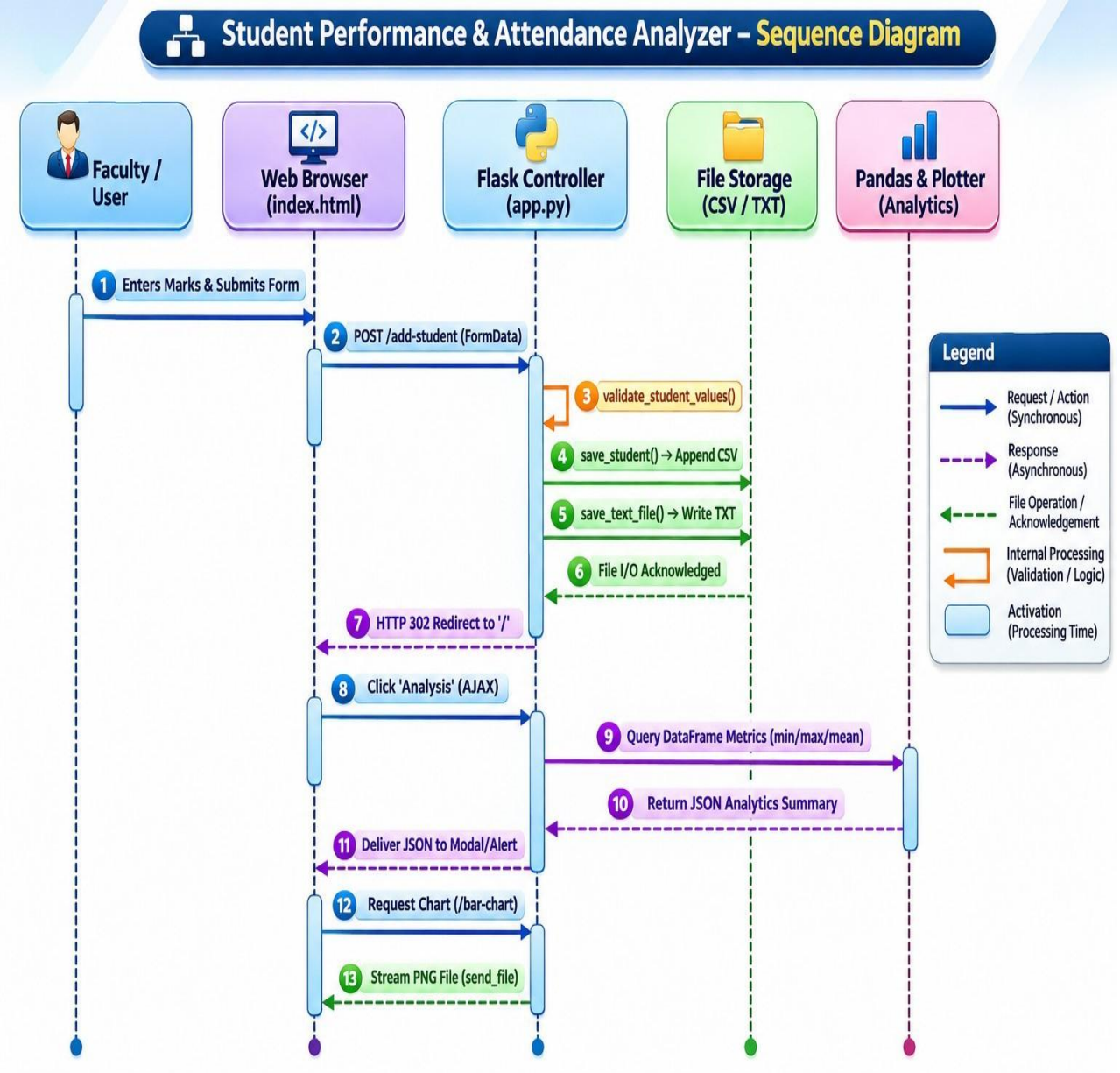


Figure 5.6: UML Sequence Diagram Illustrating Time-Ordered Message Exchanges

5.7 Flow Chart

The System Flow Chart provides a detailed algorithmic roadmap of the program execution logic from `app.py` startup to termination. It details initialization checks (folder creation for `data/` and `charts/`, CSV header creation), loop iteration over records, branching conditions based on user input, validation checks, error handling with user alert rendering, and seamless looping back to the dashboard.

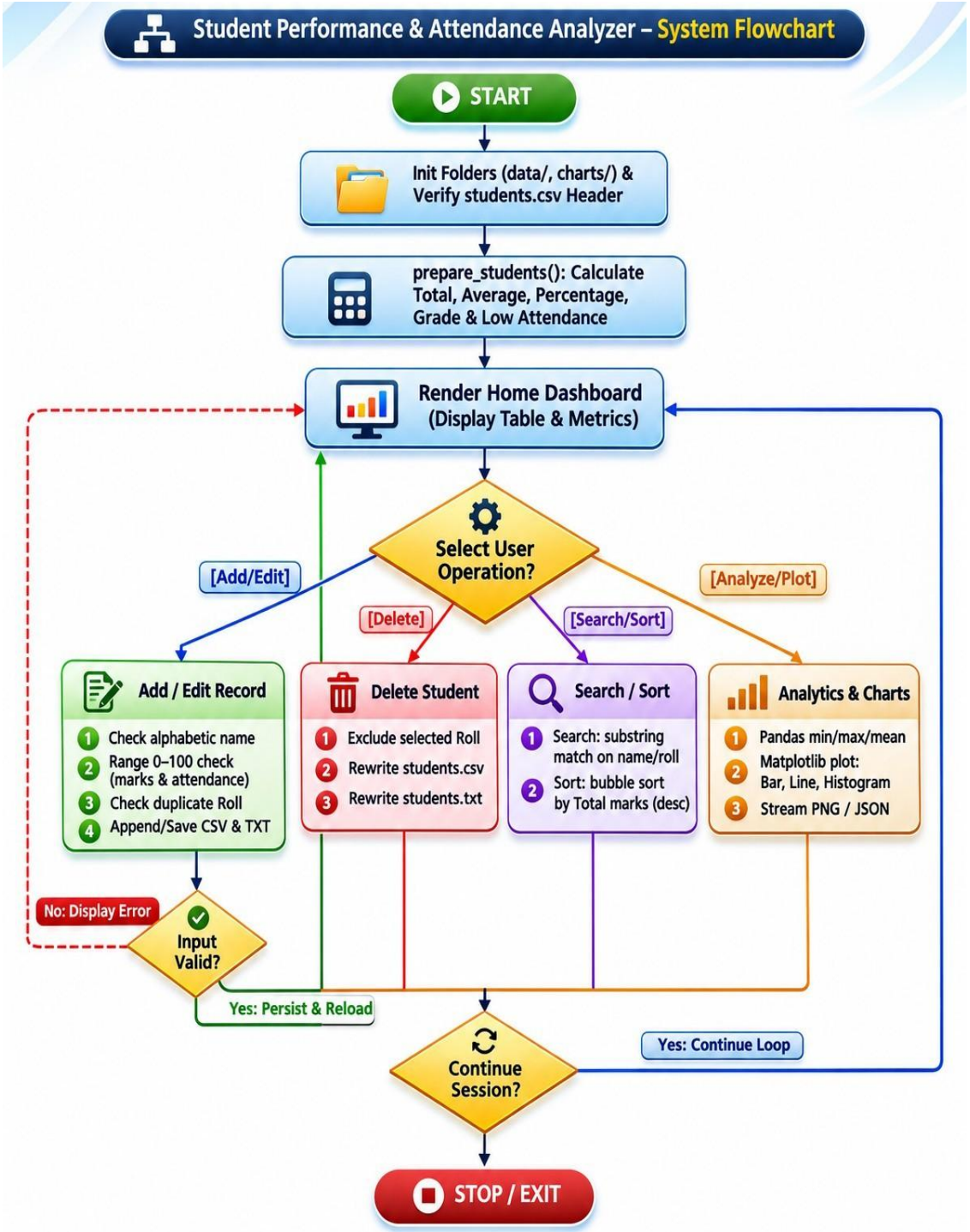


Figure 5.7: Detailed System Logic Flow Chart

6. PROPOSED DESIGN

The proposed design adopts a lightweight Model-View-Controller (MVC) architectural pattern tailored for Python and Flask. This separation of concerns ensures that data persistence, computational logic, and user presentation remain cleanly decoupled, making the application easy to maintain, debug, and expand.

6.1 System Architecture Overview

- **Model Layer (Data & Persistence):** Consists of flat file structures: `data/students.csv` for comma-delimited structured records and `data/students.txt` for human-readable audit logs. Python's built-in `csv` module manages structured read/write operations.

- **Controller Layer (`app.py`):** The central Flask application handles HTTP routing, extracts form parameters, invokes data validation routines, executes calculation formulas, controls sorting and search logic, and interfaces with Pandas and Matplotlib.

- **View Layer (`templates/index.html`):** Rendered dynamically using Flask's native Jinja2 templating engine. The UI utilizes CSS flexbox and grid layouts for responsive display, color-coded badges for grades and low attendance warnings, and vanilla JavaScript for dynamic search and modal popups.

6.2 Database Schema & File Structure

The application maintains two synchronized data stores. The CSV structure serves as the primary machine-readable datastore:

Field Name	Data Type & Constraint	Field Purpose & Description
Roll	String (max 30 chars)	Unique student identification roll number (e.g., '011')
Name	String (max 100 chars)	Full name of student; alphabetic characters and spaces only
Python	Integer (0 - 100)	Marks secured in Python Programming course
C Programming	Integer (0 - 100)	Marks secured in C Programming course
DBMS	Integer (0 - 100)	Marks secured in Database Management Systems course
Web Development	Integer (0 - 100)	Marks secured in Web Development course
Attendance	Float (0.0 - 100.0)	Cumulative attendance percentage recorded for the semester

Additionally, `data/students.txt` maintains an audit trail formatted as human-readable key-value lines:

Sample TXT Log: Roll: 011, Name: Debajyoti Bayen, Python: 84, C: 85, DBMS: 60, Web: 88, Attendance: 82.0
Roll: 012, Name: Trisha Biswas, Python: 90, C: 99, DBMS: 85, Web: 90, Attendance: 89.0

6.3 Algorithmic Implementations

- **Grade Calculation Logic:** Standardized institutional grading rules are implemented via `calculate_grade()`: A+ (>= 90%), A (>= 80%), B (>= 70%), C (>= 60%), D (>= 50%), and F (< 50%).
- **Attendance Defaulter Detection:** Evaluated during data preparation using `if student['attendance'] < 75: low_attendance.append(student)`. Students flagged here are surfaced in a prominent alert card.
- **Descending Bubble Sort:** Implemented in the `/sort` route using nested loops comparing adjacent elements: `if students[i]['total'] < students[j]['total']: swap(students[i], students[j])`.
- **Substring Search Algorithm:** Implemented in `/search` using normalized lowercase comparisons: `if keyword in name or keyword in roll: search_results.append(student)`.
- **Server-Side Chart Generation:** Matplotlib plots figures using the `Agg` headless canvas, saves them to `charts/`, and returns them to the browser via Flask's `send_file()`.

7. EXPERIMENTAL RESULT

The Student Performance & Attendance Analysis System was tested using an active test dataset comprising six student records representing a wide range of academic scores and attendance percentages. The following sections document the operational results across every stage of execution.

7.1 Data Loading Stage

Upon startup, the system loads all student records from `data/students.csv` into memory through `prepare_students()`. To ensure optimal readability and eliminate cramped table cells, the loaded data is presented in two structured companion tables:

Table 7.1A displays the primary course marks submitted and loaded from `data/students.csv` across all four technical subjects:

Roll	Student Name	Python	C Prog.	DBMS	Web Dev.
011	Debajyoti Bayen	84	85	60	88
012	Joy Chatterje	90	99	85	90
013	Neel Das	66	54	62	59
014	Milan Chakrabarty	85	74	75	68
015	Subha Saha	50	63	45	85
016	Joyti Karmakar	85	96	74	65

Table 7.1B presents the calculated evaluation metrics produced by the calculation engine, including Total Marks (out of 400), Average Marks, Percentage, Assigned Letter Grade, Attendance Percentage, and Compliance Status:

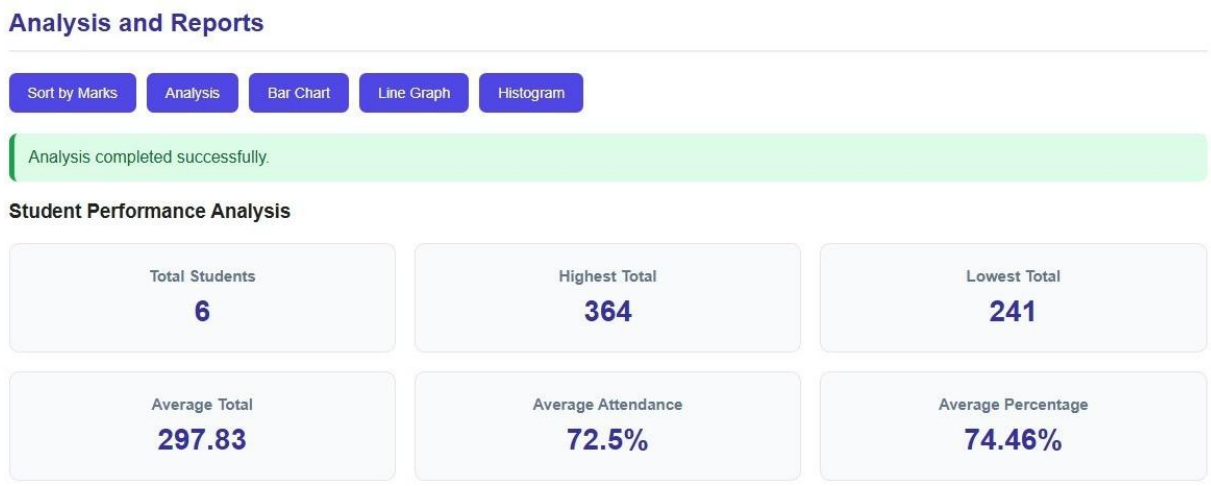
Roll	Student Name	Total	Avg	Pct (%)	Grade	Att (%)	Compliance Status
011	Debajyoti Bayen	317	79.25	79.25%	B	82.0%	Eligible (Normal)
012	Joy chatterje	364	91.00	91.00%	A+	89.0%	Eligible (Normal)
013	Neel Das	241	60.25	60.25%	C	60.0%	Defaulter (<75%)
014	Milan Chakrabarty	302	75.50	75.50%	B	80.0%	Eligible (Normal)
015	Subha Saha	243	60.75	60.75%	C	74.0%	Defaulter (<75%)
016	Joyti Karmakar	320	80.00	80.00%	A	50.0%	Defaulter (<75%)

7.2 Performance Analysis (Statistical Aggregation)

The system's statistical analysis engine was invoked via the `/pandas-analysis` endpoint. The records were ingested into a Pandas DataFrame (`pd.DataFrame(data)`), and descriptive metrics were calculated across total marks, percentages, and attendance values.

Statistical Metric	Evaluated Value	Remarks & Context
Highest Total Marks in Cohort	364 / 400	Joy (Roll 012) - Grade A+
Lowest Total Marks in Cohort	241 / 400	Neel das (Roll 013) - Grade C
Class Average Total Marks	297.83 / 400	Cohort Mean Score across all 6 students
Class Average Percentage	74.46%	Cohort Mean Academic Performance
Class Average Attendance	72.50%	Mean attendance level across all enrolled students
Total Student Enrollment	6 Students	Active records evaluated in dataset

The statistical findings reveal that while the academic performance of the cohort is solid with a mean percentage of 74.46% and a high score of 91.00%, the cohort-wide attendance average stands at 72.50%, which falls slightly below the institutional compliance threshold of 75%.



7.3 Attendance Analysis (Defaulter Identification)

Institutional policy mandates that all enrolled students maintain an attendance level of at least 75.0% to be eligible for end-semester examinations. The system automatically scans the dataset and isolates all students falling below this threshold into a dedicated alert roster.

Roll	Student Name	Recorded Att.	Shortfall	Actionable Academic Recommendation
013	Neel Das	60.0%	-15.0%	Critical Defaulter; requires formal counseling
015	Subha saha	74.0%	-1.0%	Marginal Defaulter; eligible for recovery sessions
016	Joyti Karmakar	50.0%	-25.0%	Severe Defaulter; high marks contrast with low attendance

This analysis demonstrates an important insight: student Joyti Karmakar (Roll 016) achieved strong academic marks (Total: 320, Percentage: 80.0%, Grade: A), yet possessed an attendance rate of only 50.0%. Automated defaulter detection ensures that high academic marks do not inadvertently mask critical attendance violations.



7.4 Bar Chart Generation

The `/bar-chart` route generates a comparative bar chart plotting each student's name along the X-axis against their Total Marks (out of 400) along the Y-axis. Matplotlib visualizes each student's cumulative score, providing an immediate visual comparison across the entire class cohort.

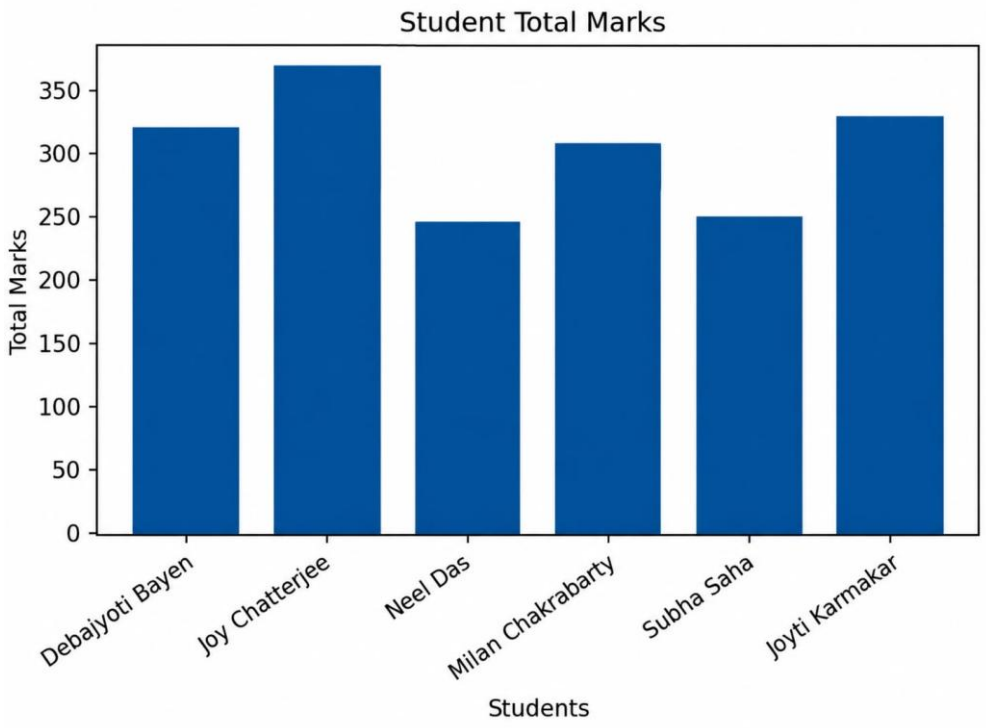


Figure 7.4: Experimental Bar Chart - Total Marks Comparison Across Students

As illustrated in Figure 7.4, Joy Chatterjee achieves the peak bar height with 364 marks, followed by Joyti Karmakar (320 marks) and Debajyoti bayen (317 marks). Neel Das and Subha Saha register closely matched totals of 241 and 243 marks respectively.

7.5 Line Graph Generation

The `/line-graph` endpoint generates a continuous performance trend line plotting students along the X-axis against their overall Percentage scores along the Y-axis. Point markers (`marker='o'`) clearly highlight each student's individual percentage level.

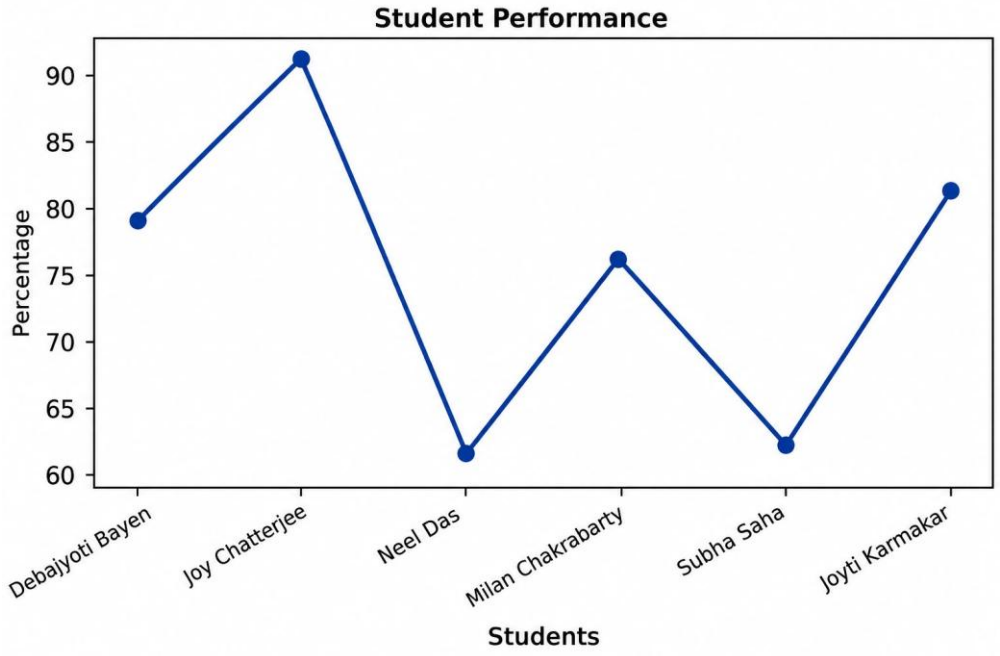


Figure 7.5: Experimental Line Graph - Student Percentage Trajectory

Figure 7.5 clearly highlights academic variation across the cohort, showing a sharp upward peak at Joy Chatterjee(91.0%), followed by a dip toward Neel Das(60.25%), a recovery to Milan Chakrabarty (75.5%), and a secondary peak at Joyti Karmakar (80.0%).

7.6 Histogram Generation

The `/histogram` endpoint constructs a frequency distribution histogram dividing percentage scores into 5 equal bins. This visualization displays the density distribution of student performance, showing how many students fall into distinct academic brackets.

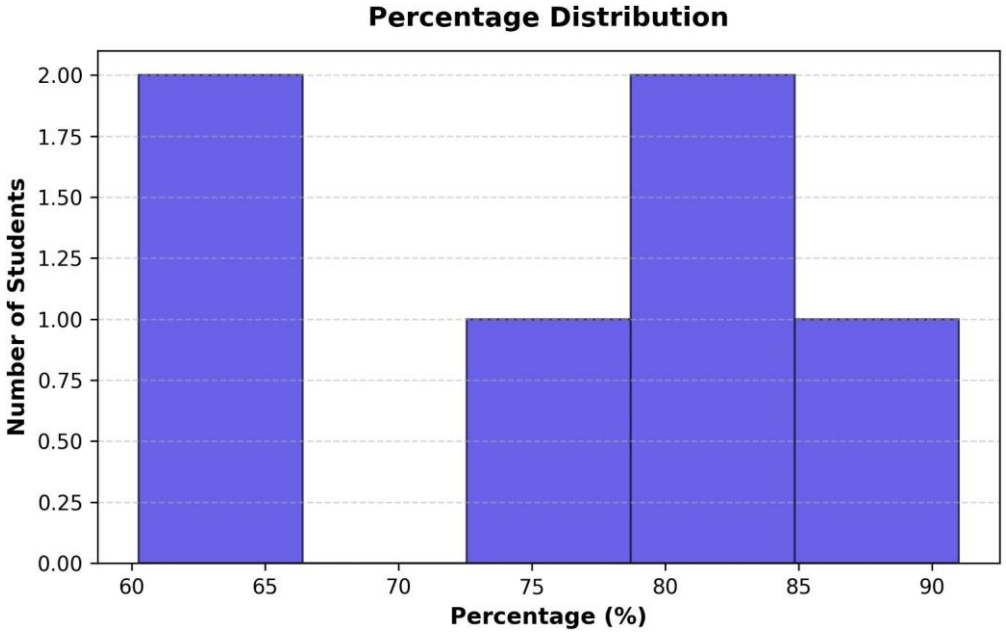


Figure 7.6: Experimental Histogram - Percentage Frequency Distribution

Figure 7.6 illustrates that student performance is distributed across two primary clusters: two students fall within the 60%-65% bracket, three students fall within the 75%-82% range, and one outstanding student achieves the 90%-95% band.

8. FUTURE SCOPE

While the current version of the Student Performance & Attendance Analysis System successfully meets all immediate functional requirements, several promising avenues exist for future enhancement and institutional scaling:

- **Migration to Scalable Relational Database (RDBMS):** While CSV storage provides outstanding simplicity and portability for small to medium class sizes, migrating the storage backend to PostgreSQL or MySQL with SQLAlchemy ORM will enable atomic transaction processing, multi-threaded write locking, and support for thousands of simultaneous student records.
- **Role-Based Access Control (RBAC) & Authentication:** Implementing secure multi-role login portals for Administrators, Faculty Members, Students, and Parents. Students would be granted read-only access to their personal scorecards, while instructors retain record manipulation privileges.
- **Automated Email & SMS Notification Triggers:** Integrating SMTP or SMS gateway APIs (such as Twilio) to automatically dispatch warning notifications to students and parents whenever attendance dips below the mandatory 75% threshold.
- **Machine Learning Predictive Modeling:** Leveraging historical performance and attendance data with Scikit-Learn to train classification models (such as Random Forest or Logistic Regression) that forecast end-semester examination outcomes and proactively identify at-risk students midway through the term.
- **Automated PDF Report Card Generation:** Developing an automated PDF export engine using ReportLab or WeasyPrint to generate formal printable semester grade sheets featuring institutional logos, teacher signatures, and personalized remarks.

9. CONCLUSION

The Student Performance & Attendance Analysis System successfully delivers a comprehensive, robust, and intuitive web-based solution for tracking, computing, and visualizing academic performance and attendance compliance. By automating computational workflows, the application completely eliminates arithmetic human errors and significantly reduces faculty workload.

Key accomplishments achieved in this project include:

- **Full-Featured CRUD & Data Integrity:** Seamless student record creation, editing, updating, and deletion with stringent validation preventing duplicate roll numbers, non-alphabetic names, and invalid score ranges.
- **Transparent Dual Persistence:** Implementation of atomic CSV storage coupled with a mirrored human-readable text audit trail, ensuring zero-configuration deployment and maximum portability.
- **Automated Defaulter Isolation:** Instantaneous detection and highlighting of students failing to meet the mandatory 75% attendance threshold, promoting timely academic intervention.
- **Statistical & Visual Insight:** One-click Pandas aggregation delivering class averages and extremes, complemented by Matplotlib Bar Charts, Line Graphs, and Histograms.

In summary, the project bridges the gap between manual record keeping and overly complicated enterprise systems, providing an elegant, efficient, and reliable analytical tool for educational institutions.